Fakultas Informatika
School of Computing
Telkom University

Aplikasi Berbasis Platform

Maps & Place

Outline

- How Location Works
- Packages for Location Services

How Location Works

Can be retrieved by these three:

Global Positioning System

Packages for Location Services:

- Geolocator
- Geocoding
- Google Maps
- Open Street Map

Geolocator

Features:

- Get the last known location
- Get the current location of the device
- Get continuous location updates
- Check if location services are enabled on the device
- Calculate the distance (in meters) between two geocoordinates
- Calculate the bearing between two geocoordinates

Geolocator

- Run on terminal (inside your project's folder) to add dependencies

```
flutter pub add geolocator
```

- Usage Example

```
Position position = await Geolocator.getCurrentPosition();
```

Geocoding

Features:

- Translate an address into latitude and longitude coordinates
- Translate latitude and longitude coordinates into an address

Geocoding

- Run on terminal (inside your project's folder) to add dependencies

```
flutter pub add geocoding
```

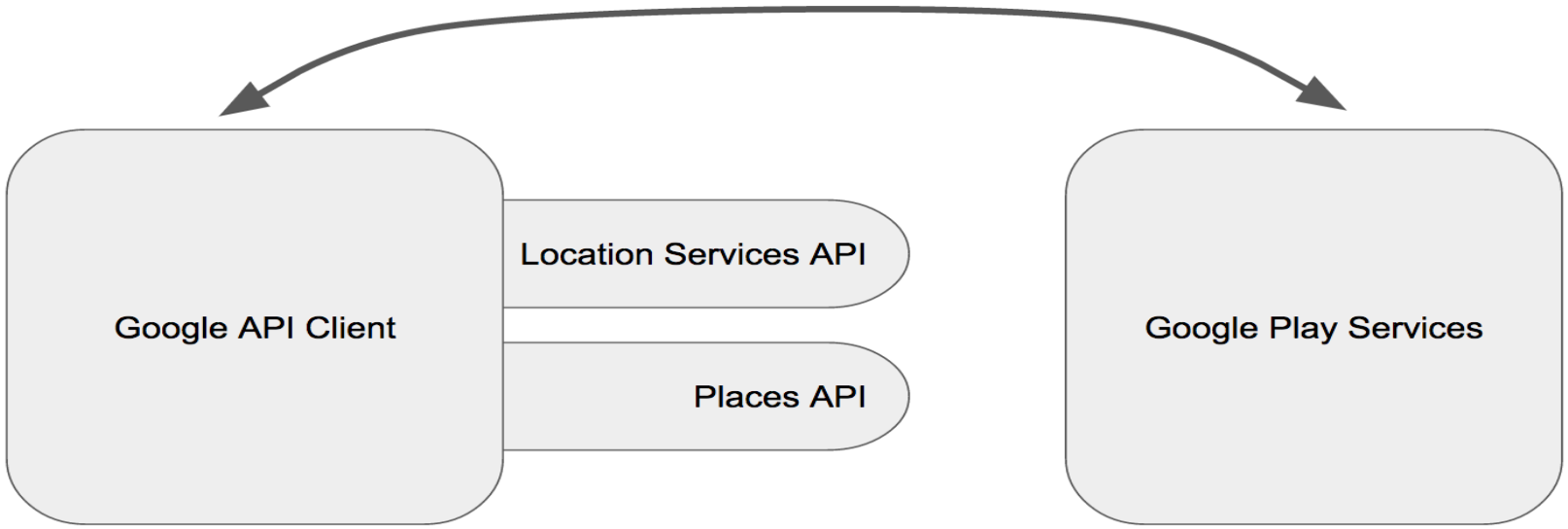
- Usage Example

```
List<Location> locations = await
locationFromAddress("Gronausestraat 710, Enschede");

List<Placemark> placemarks = await
placemarkFromCoordinates(52.2165157, 6.9437819);
```

Google Maps

- Add Google Maps Dependencies
- Putting a Map on Screen
- Add Marker on Google Maps

Google API Client for Location



Add Google Maps Dependencies

- Run on terminal (inside your project's folder)

```
flutter pub add google_maps_flutter
```

- Add dependency for web

```
flutter pub add google_maps_flutter_web
```

Adding an API key from Google

- Go to https://console.cloud.google.com/
- Create a new project or go to previous project
- Go to ‘Enabled APIs & services’ menu
- Click + button
- Search for ‘Maps SDK for Android’
- Follow the instructions
  - Usually needs credit card for insurance
  - It's free if the usage is less than 1000 requests / month

Adding an API key for an Android app

- To add an API key to the Android app, edit the **AndroidManifest.xml** file in **android/app/src/main**. Add a single meta-data entry containing the API key created in the previous step inside the application node.

```
<meta-data android:name="com.google.android.geo.API_KEY"
            android:value="YOUR-KEY-HERE"/>
```

android/app/src/main/AndroidManifest.xml

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.google_maps_in_flutter">
    <application
        android:label="google_maps_in_flutter"
        android:icon="@mipmap/ic_launcher">

        <!-- TODO: Add your Google Maps API key here -->
        <meta-data android:name="com.google.android.geo.API_KEY"
               android:value="YOUR-KEY-HERE"/>

        <activity
            android:name=".MainActivity"
            android:launchMode="singleTop"
            android:theme="@style/LaunchTheme"
            android:configChanges="orientation|keyboardHidden|keyboard|screenSize|smalles
            android:hardwareAccelerated="true"
            android:windowSoftInputMode="adjustResize">
```

Adding an API key for an iOS app

To add an API key to the iOS app, edit the **AppDelegate.swift** file in ios/Runner. Unlike Android, adding an API key on iOS requires changes to the source code of the Runner app. The **AppDelegate** is the core singleton that is part of the app initialization process.

ios/Runner/AppDelegate.swift

```swift
import UIKit
import Flutter
import GoogleMaps  // Add this import

@UIApplicationMain
@objc class AppDelegate: FlutterAppDelegate {
  override func application(
    _ application: UIApplication,
    didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey: Any]?
  ) -> Bool {
    GeneratedPluginRegistrant.register(with: self)

    // TODO: Add your Google Maps API key
    GMSServices.provideAPIKey("YOUR-API-KEY")

    return super.application(application, didFinishLaunchingWithOptions: launchOptions)
  }
}
```

Adding an API key for a Web app

- To add an API key to the Web app, edit the **index.html** file in web. Add a reference to the Maps JavaScript script in the head section, with your API key.

Adding an API key for a Web app

web/index.html

```
<head>
  <base href="/">

  <meta charset="UTF-8">
  <meta content="IE=Edge" http-equiv="X-UA-Compatible">
  <meta name="description" content="A new Flutter project.">

  <!-- iOS meta tags & icons -->
  <meta name="apple-mobile-web-app-capable" content="yes">
  <meta name="apple-mobile-web-app-status-bar-style" content="black">
  <meta name="apple-mobile-web-app-title" content="google_maps_in_flutter">
  <link rel="apple-touch-icon" href="icons/Icon-192.png">

  <!-- TODO: Add your Google Maps API key here -->
  <script src="https://maps.googleapis.com/maps/api/js?key=YOUR-KEY-HERE"></script>

  <title>google_maps_in_flutter</title>
  <link rel="manifest" href="manifest.json">
</head>
```

Google Maps

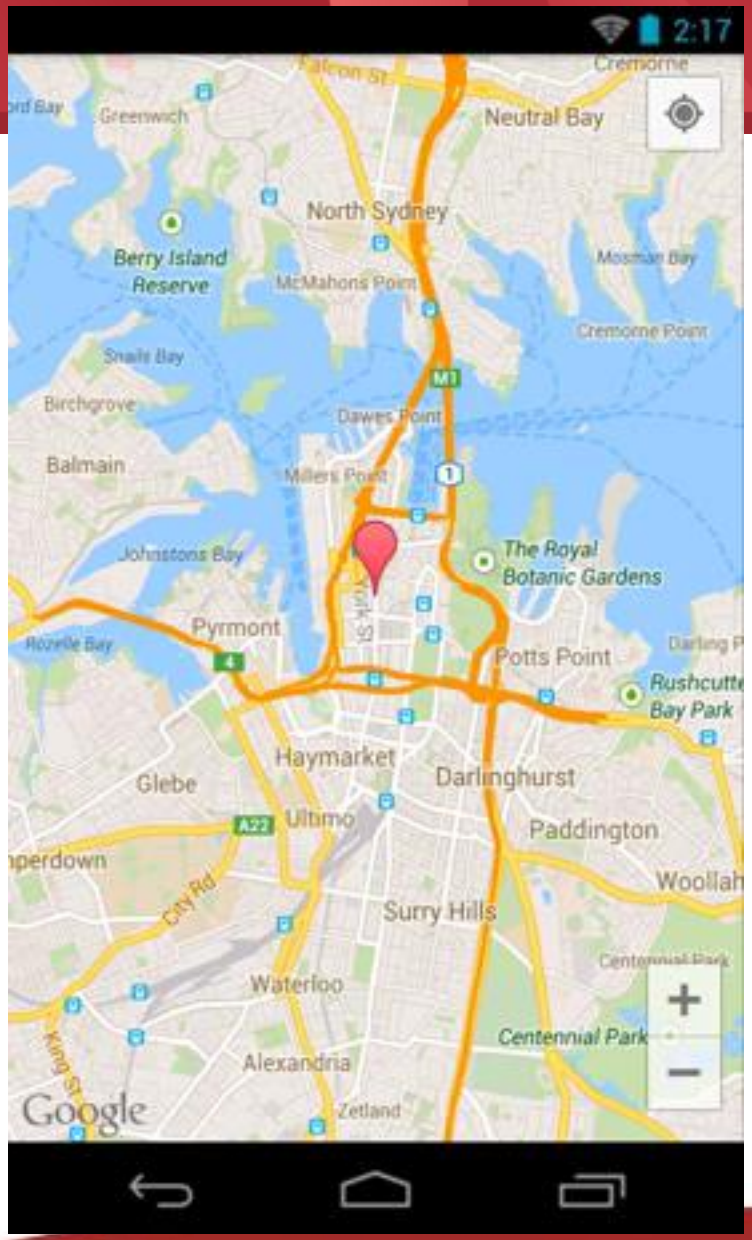
- Add Google Maps Dependencies
- Putting a Map on Screen
- Add Marker on Google Maps

Putting a Map on Screen

```
class _MyHomePageState extends State<MyHomePage> {
  late GoogleMapController mapController;

  final LatLng _center = const LatLng(-2.9547949, 104.6929238);

  void _onMapCreated(GoogleMapController controller) {
    mapController = controller;
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(widget.title),
      ), // AppBar
      body: GoogleMap(
        onMapCreated: _onMapCreated,
        initialCameraPosition: CameraPosition(
          target: _center,
          zoom: 11.0,
        ), // CameraPosition
      ), // GoogleMap
      // This trailing comma makes auto-formatting nicer for build methods.
    ); // Scaffold
  }
}
```

Map settings

- Set center
- Set zoom level
- Change map type



Google Maps

- Add Google Maps Dependencies
- Putting a Map on Screen
- Add Marker on Google Maps

Marker for Places

Position: (-37.8098, 144.9630)
Radius: 10 meters
Lifetime: 200 hours

Add Marker On Google Maps

```
List<Marker> _markers = [];

void _onMapCreated(GoogleMapController controller) {
  mapController = controller;

  setState(() {
    _markers.add(Marker(
      markerId: MarkerId("palembang"),
      position: LatLng(-2.9547949, 104.6929238),
      infoWindow: InfoWindow(
        title: "Palembang",
        snippet: "PALEMBANG EMAS",
      ), // InfoWindow
    )); // Marker
  });
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text(widget.title),
    ), // AppBar
    body: GoogleMap(
      onMapCreated: _onMapCreated,
      initialCameraPosition: CameraPosition(
        target: _center,
        zoom: 11.0,
      ), // CameraPosition
      markers: Set<Marker>.of(_markers),
    ), // GoogleMap
```

Open Street Map

- Use OSM if you don't have Google Maps API keys
- OSM has almost the same features as Google Maps

Add OSM Dependencies

- Run on terminal (inside your project's folder)

```
flutter pub add flutter_map

flutter pub add latlong2
```

Putting a Map on Screen

```
class _MyHomePageState extends State<MyHomePage> {

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: FlutterMap(
        options: MapOptions(
          initialCenter: LatLng(-6.97426, 107.6337),
          initialZoom: 11,
        ),  // MapOptions
        children: [
          TileLayer(
            urlTemplate: 'https://tile.openstreetmap.org/{z}/{x}/{y}.png',
          ),  // TileLayer
        ],
      ),  // FlutterMap
    );  // Scaffold
  }
}
```

Reference

- https://pub.dev/packages/geolocator
- https://pub.dev/packages/geocoding
- https://pub.dev/packages/google_maps_flutter
- https://pub.dev/packages/flutter_map
- https://codelabs.developers.google.com/codelabs/google-maps-in-flutter
- https://docs.fleaflet.dev

Fakultas Informatika
School of Computing
Telkom University

THANK YOU